

AI Customer Service Chatbot with Sentiment Analysis

Project 1 – Complete Technical & Operational Guide

Purpose: This document explains the project architecture, technologies, file responsibilities, execution commands, data flow, testing approach, and troubleshooting process so that another developer can understand and run the project from scratch.

1. Project Objective

The application is a real-time AI customer-service chatbot. It analyzes customer sentiment, retrieves relevant company FAQ information using semantic similarity, selects an emotion-aware response strategy, and uses Google Gemini to generate a natural response. Streamlit provides the user interface.

2. Overall Working Flow

Stage	What happens
1. Customer message	Customer enters a question or complaint in Streamlit.
2. Sentiment	RoBERTa-based sentiment model detects positive, negative, neutral or uncertain sentiment and confidence.
3. Retrieval	The customer message is converted into an embedding and compared with FAQ-section embeddings.
4. Strategy	Response strategy is selected according to detected sentiment.
5. Prompt	History, message, sentiment, strategy and retrieved company information are assembled into a Gemini prompt.
6. Generation	Gemini generates the customer-facing response.
7. UI	Streamlit displays the answer, sentiment and confidence.

3. Project Structure

```
CUSTOMER_SERVICE_CHATBOT/  
■■■ app.py  
■■■ .env  
■■■ .gitignore  
■■■ requirements.txt  
■■■ .streamlit/config.toml  
■■■ data/sentiment_test.csv  
■■■ knowledge/company_faq.txt  
■■■ src/  
■ ■■■ __init__.py  
■ ■■■ chatbot.py  
■ ■■■ embeddings.py  
■ ■■■ evaluate_sentiment.py  
■ ■■■ knowledge_base.py  
■ ■■■ response_strategy.py  
■ ■■■ retriever.py  
■ ■■■ sentiment.py  
■ ■■■ similarity.py  
■■■ tests/  
■■■ __init__.py  
■■■ test_response_strategy.py  
■■■ test_retriever.py  
■■■ test_sentiment.py
```

4. File Responsibilities

File	Responsibility
app.py	Streamlit interface, session history, chat messages and sentiment display.
src/chatbot.py	Main orchestration: sentiment → strategy → retrieval → Gemini.
src/sentiment.py	Loads and runs sentiment classification.
src/embeddings.py	Loads embedding model and creates vector representations.
src/retriever.py	Splits FAQ into sections, creates embeddings and retrieves relevant sections.
src/knowledge_base.py	Loads company FAQ text.
src/response_strategy.py	Defines communication behavior by sentiment.
src/evaluate_sentiment.py	Runs sentiment evaluation using the supplied test dataset.
src/similarity.py	Similarity helper functionality.
knowledge/company_faq.txt	Business/customer-service source of truth.
data/sentiment_test.csv	Sentiment evaluation dataset.
tests/	Automated tests for core components.
requirements.txt	Python dependencies.
.env	Gemini API key; keep private and out of Git.

5. Technologies Used

Python, Streamlit, Hugging Face Transformers, CardiffNLP Twitter RoBERTa sentiment model, sentence-transformer embeddings, NumPy, Google Gemini API / google-genai SDK, python-dotenv and pytest.

6. Knowledge Base and Semantic Retrieval

The company FAQ is stored in **knowledge/company_faq.txt**. The retriever splits the document into Markdown sections such as Payments, Order Delivery, Delayed Orders, Returns and Refunds. Each section is converted to an embedding. A customer query is embedded in the same vector space and compared with section embeddings. The highest-relevance sections are passed to the chatbot.

This is important because the chatbot does not need to send the entire FAQ to Gemini for every question. It first finds the most relevant information.

7. Sentiment Analysis

The sentiment module uses a pretrained RoBERTa-based sentiment model. The result contains a sentiment label and confidence. The sentiment is then used by `response_strategy.py` and included in the Gemini prompt.

Example: an angry message should receive acknowledgement and empathy, while a neutral factual question should receive a direct professional answer.

8. Gemini Response Generation

`chatbot.py` assembles the following information: conversation history, current customer message, detected sentiment, confidence, response strategy and retrieved FAQ content. Gemini receives this context plus a system instruction that prohibits inventing order status, tracking data, refund approval, prices, discounts, policies or delivery dates.

9. Setup From Scratch

Open Anaconda Prompt or Command Prompt and run:

```
cd /d D:\Internship\CUSTOMER_SERVICE_CHATBOT
conda activate customer_service_bot
python --version
python -m pip --version
python -m pip install -r requirements.txt
```

Create the project-root .env file:

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

The .env file must not be committed to GitHub.

10. Run the Project

```
cd /d D:\Internship\CUSTOMER_SERVICE_CHATBOT
conda activate customer_service_bot
streamlit run app.py
```

Open <http://localhost:8501> in the browser.

11. Test Components Before the UI

Retriever test:

```
python -m src.retriever
```

Direct chatbot test:

```
python -c "from src.chatbot import generate_response; print(generate_response('How long does delivery take?'))"
```

Automated tests:

```
python -m pytest -q
python -m pytest -v
```

12. Example End-to-End Interaction

Customer: My order has been delayed and I am extremely frustrated.

Sentiment: Negative, with high confidence.

Retrieved information: Delayed Orders and Order Delivery sections.

Strategy: acknowledge frustration, apologize when appropriate, show empathy and provide a useful next step.

Gemini: generates a natural customer-service response without inventing the current location or tracking status.

13. Current Known Warnings / Errors

Hugging Face unauthenticated warning: the project can still load the model; authentication mainly affects Hub access/rate limits.

RoBERTa pooler UNEXPECTED keys: the observed load report states these can be ignored for the different task/architecture loading scenario.

Gemini 429 RESOURCE_EXHAUSTED: this means the configured Gemini free-tier request quota was exceeded. It is an API quota issue, not a retriever or Streamlit failure. The project should handle it gracefully and the user should wait for the quota reset or use an appropriate quota/billing plan.

14. What Should Be Kept and What Can Be Deleted

Keep all source, configuration, knowledge, data and test files listed in the README. The following generated/cache folders are safe to delete and will be recreated when needed:

```
__pycache__/
.pytest_cache/
```

Do not delete src/__init__.py or tests/__init__.py if they are being used to keep the directories as Python packages. Do not delete .gitignore, requirements.txt, knowledge/company_faq.txt, data/sentiment_test.csv, app.py or the src modules.

15. Recommended Final Validation

- Run `python -m pytest -q`

- Run `python -m src.retriever`
- Run a direct `generate_response` test when Gemini quota is available
- Run `streamlit run app.py`
- Test positive, neutral and negative customer messages
- Test an order-specific question and confirm the bot does not invent tracking data
- Verify `.env` is excluded by `.gitignore`
- Capture screenshots for the internship submission
- Keep README and this technical guide with the final project

16. Final Architecture Summary

```

Customer
↓
Streamlit (app.py)
↓
generate_response() (chatbot.py)
  ■■■ sentiment.py
  ■■■ response_strategy.py
  ■■■ retriever.py
    ■ ■■■ knowledge_base.py
    ■ ■■■ embeddings.py
    ■■■ Google Gemini
    ↓
  Final customer-service response
    ↓
  Streamlit display

```

17. Project Value

This project demonstrates a practical GenAI application rather than a standalone chatbot. It combines NLP sentiment classification, semantic retrieval, controlled prompting, a generative model, business knowledge and a user-facing web application. The separation of components also makes the system easier to test and extend.